

Programación de Sistemas

Unidades 1 y 2: Estructura del compilador y análisis léxico

Cuadernillo de ejercicios 02

Del software de sistemas al análisis léxico

Versión para docente

Manuel Soto Romero

Araceli Liliana Reyes Cabello

Semestre 2026-2

Colegio de Ciencia y Tecnología UACM

Este cuadernillo acompaña las notas 02 y 03. Su propósito es practicar la estructura general de un compilador, la función y comunicación de sus fases y los fundamentos del análisis léxico. Las actividades avanzan desde identificar entradas y salidas hasta seguir la transformación de caracteres en tokens y distinguir token, lexema, patrón y atributo.

Observación 1. Límite de trabajo

Responde únicamente con lo estudiado en las notas 02 y 03. Los fragmentos de código son ilustrativos: no definen la sintaxis de IMP. No necesitas escribir expresiones regulares, construir autómatas, proponer reglas oficiales de IMP ni diseñar estrategias de recuperación de errores.

Observación 2. Ruta sugerida

Los ejercicios **1, 3, 5, 7, 8 y 9 son esenciales**: cubren los contratos del compilador y la transformación de caracteres en tokens. Los ejercicios **2, 4, 6 y 10 son de extensión**: profundizan o integran lo aprendido. Si el tiempo es limitado, completa primero los esenciales; ningún ejercicio se elimina del cuadernillo.

1. Estructura y fases del compilador

Ejercicio 1. Reconocer los contratos entre fases

[Esencial]

Completa la columna de salida con las expresiones del banco. Cada expresión se usa una sola vez.

programa destino	representación estructurada	secuencia de tokens
representación intermedia equivalente	representación validada	representación intermedia

Fase	Entrada principal	Salida
Análisis léxico	Caracteres del programa fuente	
Análisis sintáctico	Secuencia de tokens	
Análisis semántico	Representación estructurada e información necesaria	
Generación de representación intermedia	Representación validada	
Optimización	Representación intermedia	
Generación de código	Representación validada o intermedia	

Respuesta 1

En orden: secuencia de tokens; representación estructurada; representación validada; representación intermedia; representación intermedia equivalente; programa destino.

Ejercicio 2. Análisis o síntesis

[Extensión]

Escribe **A** si la tarea corresponde a la etapa general de análisis y **S** si corresponde a la etapa general de síntesis.

Tarea	A/S
Agrupar caracteres para producir tokens.	
Reconocer la estructura de una secuencia de tokens.	
Comprobar restricciones de declaración y tipos.	
Construir una representación intermedia a partir de una representación comprobada.	
Transformar una representación para mejorar una característica sin cambiar el comportamiento definido.	
Producir instrucciones o construcciones del lenguaje destino.	

Explica por qué la representación producida por el análisis es necesaria para iniciar la síntesis:

Respuesta 2

Orden esperado: A, A, A, S, S, S.

La representación producida por el análisis comunica una versión estructurada y comprobada del programa. La síntesis utiliza esa información para transformar el programa sin tener que volver a interpretar directamente el flujo original de caracteres.

2. Responsabilidades y diagnósticos

Ejercicio 3. ¿Qué fase debe responder?

[Esencial]

Escribe **L** para análisis léxico, **S** para análisis sintáctico o **M** para análisis semántico.

Situación	L/S/M
Ningún patrón de token permite reconocer el inicio de los caracteres restantes.	
Los tokens fueron reconocidos, pero su secuencia no puede organizarse de acuerdo con la gramática.	
Un identificador aparece en una estructura válida, pero no satisface una regla de declaración.	
Dos operandos forman una expresión sintácticamente válida, pero sus tipos no son compatibles con la operación.	
Una secuencia de caracteres se reconoce como un identificador.	
Una secuencia de tokens se reconoce como una asignación.	

¿Por qué el analizador léxico no debe decidir por sí solo si un identificador fue declarado correctamente?

Respuesta 3

Orden esperado: L, S, M, M, L, S.

El analizador léxico reconoce y clasifica el lexema como identificador. Comprobar una declaración requiere considerar la estructura del programa y la información semántica disponible; por tanto, corresponde a una fase posterior.

Ejercicio 4. Delimitación de responsabilidades

[Extensión]

Marca cada afirmación como **verdadera** o **falsa**. Corrige las afirmaciones falsas.

1. El analizador sintáctico recibe una secuencia de tokens. V ☐ F ☐
2. El analizador semántico comprueba restricciones que no dependen únicamente de la forma gramatical. V ☐ F ☐
3. El analizador léxico determina la estructura completa de las expresiones. V ☐ F ☐
4. La optimización puede cambiar el resultado del programa si con ello reduce su tamaño. V ☐ F ☐
5. La generación de código produce una representación en el lenguaje destino. V ☐ F ☐
6. Todos los compiladores deben tener exactamente una representación intermedia y un optimizador. V ☐ F ☐

Respuesta 4

1. Verdadera. 2. Verdadera. 3. Falsa. 4. Falsa. 5. Verdadera. 6. Falsa.

Correcciones:

- ★ El analizador **sintáctico** determina cómo se organizan los tokens en estructuras; el léxico reconoce unidades a partir de caracteres.
- ★ La optimización debe conservar el comportamiento definido del programa.

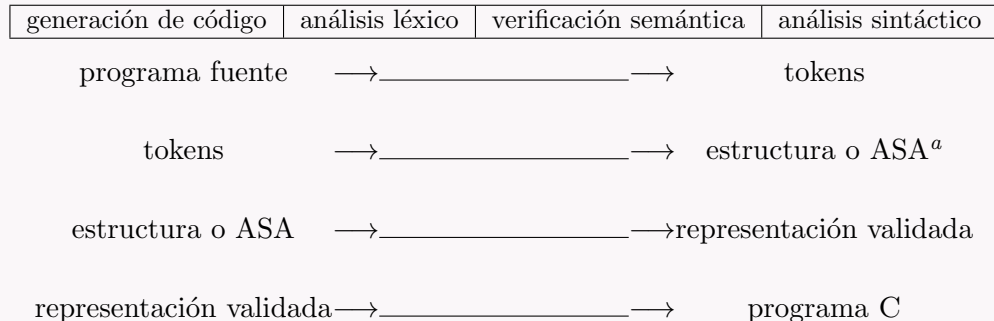
- ★ Un compilador puede usar varias representaciones intermedias, repetir optimizaciones o no incluir un optimizador en una implementación sencilla.

3. Integración del compilador del curso

Ejercicio 5. Completar el recorrido de IMP/IMP++ a C

[Esencial]

El recorrido interno será el mismo para el IMP base y para su extensión IMP++. Completa el flujo con las cuatro fases del banco.



Responde brevemente.

1. ¿Qué versión construiremos en notas y clase, y cuál desarrollaremos en las prácticas?
2. ¿Qué fases serán apoyadas por Lark?
3. ¿Qué fases programaremos mediante recorridos en Python?
4. ¿El programa C producido ya es un archivo ejecutable? Explica.
5. ¿Dónde puede ubicarse una representación intermedia en este recorrido?

^aASA significa *árbol de sintaxis abstracta*. La denominación proviene del inglés *Abstract Syntax Tree*, abreviado AST.

Respuesta 5

En orden: análisis léxico; análisis sintáctico; verificación semántica; generación de código.

1. En notas y clase construiremos la versión base para IMP; en las prácticas extenderemos ese producto para IMP++.
2. Lark apoyará el análisis léxico y el análisis sintáctico.
3. La verificación semántica y la generación de código se implementarán explícitamente en PYTHON mediante recorridos sobre la representación construida.
4. No. El destino de nuestro compilador es un programa en C; después debe intervenir un compilador de C para continuar la cadena hacia un ejecutable.
5. Puede ubicarse después de obtener una representación validada y antes de la generación de código. Su forma concreta se definirá en otra nota.

Ejercicio 6. Interfaces y cambios

[Extensión]

Un equipo define esta interfaz:



Responde.

1. ¿Qué debe conocer el analizador sintáctico sobre la salida del léxico?
2. Si cambia la forma de representar los tokens, ¿qué componente debe revisarse además del

analizador léxico y por qué?

3. Explica por qué una interfaz clara ayuda a probar el compilador por fases.

Respuesta 6

1. Debe conocer el formato de cada token: el nombre que representa la categoría y, cuando se requiera, el atributo comunicado.
2. Debe revisarse el analizador sintáctico porque consume esa representación. También deben revisarse otros componentes que utilicen directamente los atributos modificados.
3. Una interfaz clara permite preparar entradas controladas, observar salidas y verificar cada responsabilidad sin ejecutar necesariamente todo el compilador.

4. Fundamentos del análisis léxico

Ejercicio 7. Token, lexema, patrón o atributo

[Esencial]

Relaciona cada término con su descripción. Escribe la letra correspondiente.

Término	Descripción
1. Token	A. Información adicional que distingue la aparición reconocida, por ejemplo el texto de un identificador o el valor de un entero.
2. Lexema	B. Descripción de la forma que pueden tener los lexemas asociados con un nombre de token.
3. Patrón	C. Secuencia concreta de caracteres reconocida en el programa fuente.
4. Atributo	D. Par formado por un nombre de token y un valor de atributo opcional.

1	2	3	4

Completa las relaciones:

- ★ El patrón describe una _____.
- ★ El lexema _____ ese patrón.
- ★ El token _____ el resultado a otra fase.

Respuesta 7

1-D, 2-C, 3-B y 4-A.

Relaciones: el patrón describe una **forma**; el lexema **satisface** ese patrón; el token **comunica** el resultado a otra fase.

Ejercicio 8. Analizar un fragmento ilustrativo

[Esencial]

Considera el fragmento `total = 25;`. No supongas que pertenece a IMP. Completa la tabla con los datos del banco.

PYC	ID	ENTERO	ASIG
⟨ENTERO, 25⟩	⟨PYC⟩	⟨ID, total⟩	⟨ASIG⟩

Lexema	Nombre	Patrón informal	Token
total		Letra seguida opcionalmente de letras o dígitos.	
=		El carácter =.	
25		Secuencia de uno o más dígitos.	
;		El carácter ;.	

Responde:

1. ¿Qué tokens necesitan atributo en este ejemplo y para qué sirve?
2. ¿Por qué **total** es un lexema y no un patrón?

Respuesta 8

Por filas:

Lexema	Nombre	Patrón informal	Token
total	ID	Letra seguida opcionalmente de letras o dígitos.	⟨ID, total⟩
=	ASIG	El carácter =.	⟨ASIG⟩
25	ENTERO	Secuencia de uno o más dígitos.	⟨ENTERO, 25⟩
;	PYC	El carácter ;.	⟨PYC⟩

1. ID necesita el atributo **total** para distinguir ese identificador de otros. ENTERO necesita el atributo 25 para conservar el valor o su representación. En esta descripción, ASIG y PYC no necesitan atributo.
2. **total** es la secuencia concreta encontrada en la entrada. El patrón es la descripción general que pueden satisfacer muchos lexemas, por ejemplo “letra seguida opcionalmente de letras o dígitos”.

5. Comunicación y revisión integradora

Ejercicio 9. Solicitar el siguiente token

[Esencial]

Completa el párrafo con las expresiones del banco. Cada expresión se usa una sola vez.

atributo lexema caracteres token nombre de token

Cuando el analizador sintáctico necesita otra unidad, solicita el siguiente _____. El analizador léxico consume los _____ necesarios y los agrupa en un _____. Después devuelve un _____ y, cuando se requiere, un _____.

Responde brevemente.

1. ¿Es correcto eliminar siempre todos los espacios y saltos de línea? ¿Por qué?
2. ¿Qué debe especificar el equipo antes de que Lark pueda producir tokens?
3. ¿Cómo puede observarse el analizador léxico de manera independiente con Lark?

Respuesta 9

En orden: token; caracteres; lexema; nombre de token; atributo.

1. No. Deben aplicarse las reglas del lenguaje; en algunos lenguajes los espacios o saltos de línea pueden ser significativos.
2. Debe especificar las categorías que se reconocerán y el patrón o regla que describe los lexemas de cada una.
3. Puede utilizarse Lark con `parser=None`, `lexer="basic"` y el método `lex()` para recorrer una entrada y observar los tokens producidos.

Ejercicio 10. Detectar y corregir confusiones**[Extensión]**

Cada afirmación contiene un error. Subraya la parte incorrecta y reescribe la oración de forma adecuada.

1. Un lexema y un patrón son lo mismo: ambos son la secuencia concreta encontrada en el programa.
2. El analizador léxico recibe tokens y produce caracteres para el analizador sintáctico.
3. Si dos lexemas pertenecen al token ID, entonces deben contener exactamente los mismos caracteres.
4. La optimización puede reparar un programa sintácticamente inválido antes de generar código.
5. El programa C generado por el compilador del curso ya es directamente un ejecutable de la máquina.
6. Lark decide automáticamente qué categorías necesita el lenguaje, aunque el equipo no las especifique.

Respuesta 10

Correcciones esperadas:

1. Un **lexema** es la secuencia concreta encontrada; un **patrón** describe la forma de los lexemas de un nombre de token.
2. El analizador léxico recibe **caracteres** y produce una secuencia de **tokens** para el analizador sintáctico.
3. Dos lexemas pueden corresponder a ID porque satisfacen el mismo patrón, aunque tengan caracteres y atributos distintos.
4. La optimización recibe una representación válida y la transforma sin cambiar el comportamiento definido; no repara la sintaxis del programa fuente.
5. El programa C es el destino de nuestro compilador, pero necesita un compilador de C para continuar la cadena hacia un ejecutable.
6. El equipo debe especificar las categorías y reglas léxicas; Lark aplica esa especificación para producir tokens. Primero construiremos las reglas de IMP en clase y después agregaremos las de IMP++ en la práctica.